

Le Guilly Tanguy

1. Réponse au cahier des charges

Le projet respecte les objectifs du sujet « conception d'une architecture distribuée avec routage en oignon ».

Le projet a un master, plusieurs routeurs et des clients qui échangent des messages anonymement par un routage multi-sauts chiffré en couches.

2. Éléments implémentés ou non implémentés

Implémenté :

Routage multi-sauts entre clients

Communication réseau en Python avec sockets TCP et gestion multi-threads sur le master, les routeurs et les clients.

Génération locale de paires de clés RSA sur chaque routeur et chiffrement asymétrique couche par couche côté client.

Base de données MariaDB pour stocker les routeurs (IP, port, clé publique) et les clients (pseudo, IP, port).

Interfaces graphiques pour le master et pour les clients

Non implémenté ou partiel :

Pas de client qui joue aussi le rôle de routeur

3. Structure du code, modules, protocole, API

3.1 Structure des fichiers

master.py : serveur maître, connexion MariaDB, interface graphique

routeur.py : génération de clés, déchiffrement d'une couche, relais vers le prochain nœud et interface graphique.

client.py : client graphique, enregistrement au master, construction du message en oignon, envoi et réception.

3.2 Modules principaux

Sockets / Threading : socket, threading pour toutes les communications et serveurs.

GUI : PyQt6.

BDD : mysql.connector pour la connexion à MariaDB depuis le master.

3.3 Protocole applicatif

Messages simples sur TCP, encodés en UTF-8, avec un séparateur || :

Récupération des routeurs :

GET_NODES → réponse : ip:port:cle;ip:port:cle;...

Récupération des clients :

GET_CLIENTS → réponse : pseudo:ip:port;...

Routage :

RELAY||DONNEES_CHIFFREES

4. Algorithme de chiffrement (forces / faiblesses)

4.1 Principe

Chaque routeur génère une paire de clés RSA

(e,n)

(e,n) (publique) et

(d,n)

(d,n) (privée) à partir de deux nombres premiers choisis entre 100 et 500.

Le routeur envoie seulement sa clé publique au master, qui la stocke et la diffuse aux clients.

Le client construit un oignon de chiffrement :

Dernière couche : FINAL||IP_CLIENT_DEST||PORT_CLIENT_DEST||MESSAGE chiffrée avec la clé publique du dernier routeur.

À chaque couche vers l'extérieur, il ajoute

IP_PROCHAIN_ROUTEUR||PORT_PROCHAIN_ROUTEUR||COUCHE_CHIFFRÉE, puis chiffre le tout avec la clé publique du routeur correspondant.

Chaque routeur qui reçoit RELAY||... :

Déchiffre avec sa clé privée, lit l'ip et le port du prochain saut et renvoie RELAY||PAYLOAD_SUIVANT.

Le dernier routeur voit FINAL et donne le message au client final.

4.2 Forces

Aucun routeur ne voit l'itinéraire complet ni l'expéditeur et le destinataire en même temps.

4.3 Faiblesses

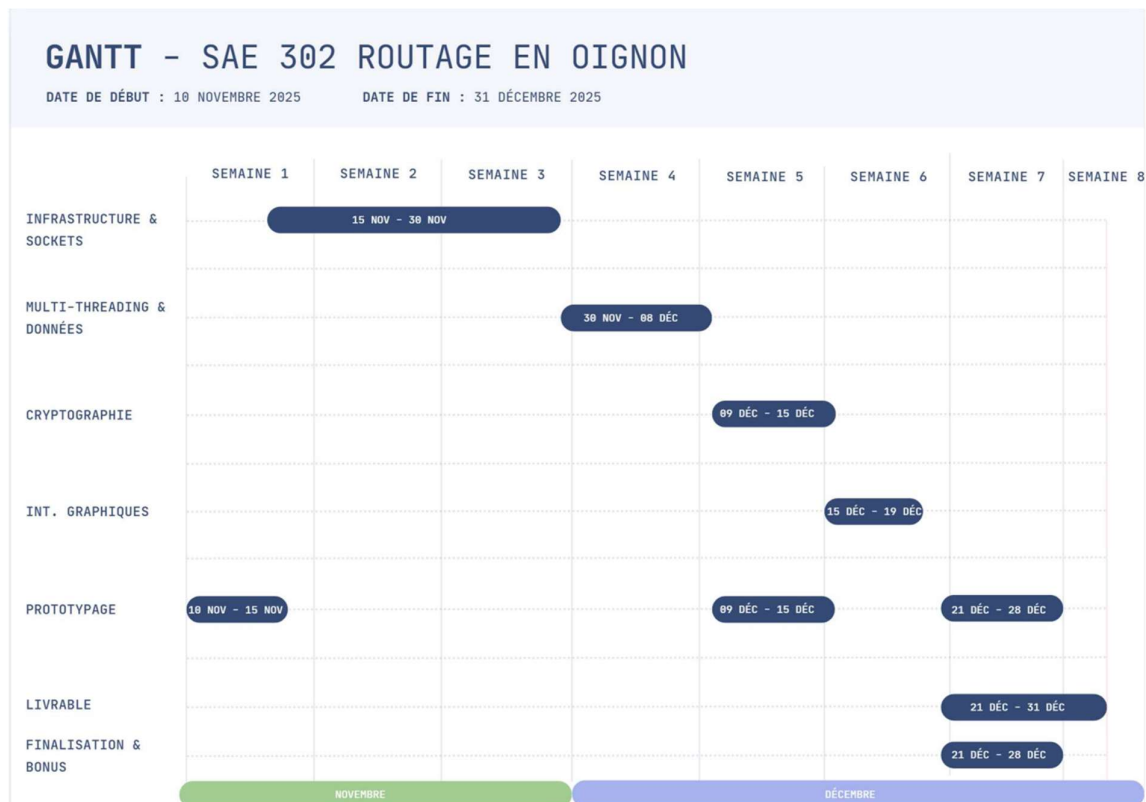
Nombres premiers de petite taille (100–500), moins sécurisé.

Pas de certificats

5. Rapport de projet et gestion du projet

Le planning Gantt fourni des étapes: sockets, threading, cryptographie, interfaces, intégration finale.

Voici le Gantt initial :



Je dirais que je l'ai plutôt bien respecter, mais j'ai quand même eu certains problèmes au niveau du chiffrement qui m'a pris beaucoup plus de temps, la partie cryptographie faisait bugger tout le projet.